

PROJET SYSTÈME C

DRAGON REACTION GAME



Un mini-jeu multijoueur en C qui dompte les **threads**, les **mutex** et les **sockets TCP**.

► PRESS START ◀

// LANGUAGE

C + POSIX

// CONCEPTS

Threads • Mutex • TCP

// JOUEURS

1 à 4 en réseau

// MISSION

Sauver le dragon... ou pas

PROBLÈME & OBJECTIF

Comment apprendre la **programmation système** sans s'endormir sur un livre ?



PROBLÈME

Trois concepts arides à dompter en même temps.

- Les **threads** et le parallélisme : invisibles, abstraits, parfois magiques.
- La **race condition** : un bug qui n'apparaît qu'au pire moment.
- Les **sockets TCP** : une API verbeuse, des erreurs cryptiques.
- Et surtout... aucune **émotion** dans un printf.



OBJECTIF

Transformer la théorie en mini-jeu jouable.

- Construire un **jeu de réflexes multijoueur** en C : un dragon qu'il faut nourrir et amuser.
- Faire **cohabiter plusieurs joueurs** sur un serveur via le réseau.
- **Voir et toucher** les notions : thread, mutex, **socket**, section critique.
- **Apprendre en jouant...** et en perdant un dragon ou deux.

CONCEPT DU JEU

Un dragon impatient. Quatre joueurs. Une seule règle : **tape le bon caractère, le plus vite possible.**



FAIM

Le dragon a faim ? Tape **F** avant les autres.
+20 de faim, +10 points pour le plus rapide.



LE DRAGON

Sa **faim** et son **humeur** baissent toutes les 3 à 6 secondes. S'il atteint 0... *game over.*



HUMEUR

Le dragon s'ennuie ? Tape **H** en premier.
+20 d'humeur, +10 points au plus rapide.

+10 pts bonne lettre & premier · -5 pts mauvaise lettre · -2 pts trop lent ou spam · le score ne descend jamais sous 0.

03 ARCHITECTURE CLIENT



SERVEUR

Une architecture **en étoile** : un serveur central, plusieurs clients connectés en TCP.

QUI FAIT QUOI ?

Le serveur est la vérité : il décide qui est arrivé en premier.

Le client est l'interface : il transforme une touche en paquet réseau.

SERVEUR

port 5555 - TCP
arbitre la partie

Gère le dragon, les joueurs,
les scores, et la **cohérence** du jeu.

POURQUOI CE CHOIX ?

Un seul endroit gère le **dragon partagé** : pas de désynchronisation entre joueurs.

Et... c'est l'occasion rêvée de manipuler des **sockets TCP** en C.

CLIENT #1

Lit le clavier,
affiche les messages,
envoie les frappes.

CLIENT #2

Même rôle,
même protocole,
connexion indépendante.

CLIENT #3

Chaque client est
un processus séparé,
parfois sur une autre machine.

CLIENT #4

Jusqu'à 4 joueurs
en simultané.
(MAX_JOUEURS = 4)

TCP - 5555

TCP - 5555

TCP - 5555

TCP - 5555

RÔLE DU SERVEUR

Le **maître du donjon** : il fait vivre le dragon, écoute les joueurs et tranche les conflits.

1

ACCUEIL

Crée un socket TCP sur le port 5555, écoute, accepte les joueurs un par un et leur demande un pseudo.

```
socket() - bind() - listen() -  
accept()
```

2

VIE DU DRAGON

Un **thread générateur** tourne en boucle : toutes les 3 à 6 secondes, il baisse **faim** et **humeur** de 10, puis lance un événement 'f' ou 'h'.

3

ARBITRAGE

Un **thread par joueur** reçoit les frappes. Le **mutex verrou_jeu** garantit qu'un seul joueur à la fois est jugé : bonne lettre, mauvaise lettre ou trop lent.

4

FIN & SCORE

Quand le dragon meurt, le serveur ferme les sockets, attend tous les threads et diffuse le **classement final** trié.

```
$ gcc server.c -o server -lpthread + $ ./server // 1 thread générateur + N threads joueurs + 1 mutex partagé
```


ROLE DU CLIENT

Léger, réactif, à **deux mains** : il tape... et il écoute, en même temps.

DEUX THREADS, UN SEUL JOUEUR

THREAD PRINCIPAL

Lit le clavier avec `fgets` puis `send` au serveur. Boucle tant que la partie est en cours.

THREAD D'ÉCOUTE

`recv` en boucle, affiche les messages du serveur (événements, scores, classement final).



Les deux threads partagent le même socket : l'un envoie, l'autre reçoit. **Aucun blocage** : on tape pendant que l'écran s'actualise.

EXTRAIT - client.c

```
while (connecte) {
    int n = recv(sock_global, buf, sizeof(buf)-1, 0);
    if (n <= 0) {
        connecte = 0;
        fclose(stdin); // débloque fgets() du main
        break;
    }
    printf("%s", buf);
}
```

L'ASTUCE - fclose(stdin)

Quand le serveur coupe la connexion, `fgets` attendrait pour toujours une touche.

Le thread d'écoute ferme alors `stdin` : `fgets` renvoie NULL, le main sort proprement, le programme termine.

```
$ gcc client.c -o client -lpthread + $ ./client // IP du serveur, pseudo, et c'est parti.
```

THREADS & PARALLÉLISME

Tout le monde travaille **en même temps**. C'est ce qui rend le jeu vivant — et risqué.

CARTE DES THREADS — CÔTÉ SERVEUR

THREAD GÉNÉRATEUR

sleep(3-6s) → faim -= 10 · humeur -= 10 → tire 'f' ou 'h' → diffuse

THREAD JOUEUR #1

recv() → juge la frappe → met à jour score & santé → diffuse le résultat

THREAD JOUEUR #2

... même logique, en parallèle, sans se soucier des autres.

THREAD JOUEUR #N

jusqu'à MAX_JOUEURS = 4 threads simultanées.

VARIABLES PARTAGÉES (mémoire commune)

faim · humeur · jeu_termine · lettre_active · joueurs[].score

POURQUOI DES THREADS ?

Le serveur ne peut pas attendre un seul joueur.
Pendant que J1 réfléchit, le dragon **vit** et J2 **tape**.
Chaque thread fait *une seule* chose, et toutes ces choses arrivent en même temps.

EXTRAIT — server.c

```
pthread_create(&thread_gen,  
              NULL, thread_générateur, NULL);  
  
for (int i = 0; i < nb_max; i++) {  
    ids[i] = i;  
    pthread_create(&threads_j[i],  
                  NULL, gener_joueur, &ids[i]);  
}
```

→ N+1 threads qui partagent la même mémoire... il va bien falloir mettre de l'ordre.

MUTEX vs RACE CONDITION

Deux joueurs, une même seconde, un seul gagnant. **verrou_jeu** tranche le duel.

SANS MUTEX — CHAOS

*Les deux threads lisent **Lettre_active** = 'f' en même temps...*

```
[t = 0.000s] J1 lit lettre_active + 'f' ✓  
[t = 0.001s] J2 lit lettre_active + 'f' ✓  
[t = 0.002s] J1 fait score += 10  
[t = 0.003s] J2 fait score += 10  
[t = 0.004s] Les deux pensent avoir gagné.  
[t = 0.005s] faim devient incohérente.
```

→ RACE CONDITION : le bug du pire moment.

Le résultat dépend de l'**ordre exact** des threads. Et cet ordre... on ne le maîtrise pas.

AVEC MUTEX — ORDRE

Un seul thread à la fois entre dans la section critique.

```
pthread_mutex_lock(&verrou_jeu);  
  
if (lettre_active == 'f' && touche == 'f') {  
    joueurs[id].score += 10;  
    faim = (faim + 20 > 100) ? 100 : faim + 20;  
    lettre_active = ' '; // consommé  
} else if (lettre_active != ' ') {  
    joueurs[id].score -= 5; // mauvaise lettre  
} else {  
    joueurs[id].score -= 2; // trop lent / spam  
}  
  
pthread_mutex_unlock(&verrou_jeu);
```

Le 1^{er} verrouille, gagne, remet ' '. Le 2nd arrive : la lettre est **consommée** → trop tard.

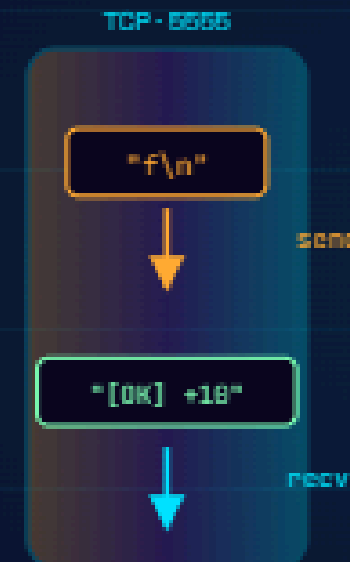
→ Le mutex protège : faim · humeur · jeu_termine · lettre_active · scores. Un seul verrou pour tout l'état partagé.

SOCKETS TCP

Le tuyau qui fait voyager une touche du clavier d'un joueur jusqu'à la mémoire du serveur — et retour.

CÔTÉ SERVEUR

```
① socket(AF_INET, SOCK_STREAM, 0)
② setsockopt(SO_REUSEADDR)
③ bind(socket, port 5555)
④ listen(socket, nb_max)
⑤ accept() // boucle pour chaque joueur
⑥ recv() / send() // pseudo, frappes, diffusion
⑦ shutdown() + close() // fin de partie
```



CÔTÉ CLIENT

```
① socket(AF_INET, SOCK_STREAM, 0)
② inet_pton(IP du serveur)
③ connect(socket, adresse)
④ recv/send du pseudo
⑤ pthread_create(thread d'écoute)
⑥ Boucle fgets + send // frappes
⑦ pthread_join + close
```

TCP = flux fiable, ordonné. Pas besoin de gérer les pertes ou l'ordre des paquets.

Ce qui passe dans le tuyau ? De simples **chaînes texte** : "f", "[OK] Lina +10", "=== CLASSEMENT ===".

LE DRAGON, EN VRAI

Deux jauges, un tirage aléatoire, une règle de mort. Pas plus, pas moins.

SANTÉ DU DRAGON

FAIM



HUMEUR



Tic-tac toutes les 3-6 s : `faim -= 10` · `humeur -= 10`. Une bonne réponse rajoute +20 (plafonné à 100).

CONDITION DE MORT

Si `faim ≤ 0` OU `humeur ≤ 0` :

`jeu_termine = 1`, le thread générateur sort de sa boucle.

Le main ferme alors les sockets et lance le classement.

★ LE DRAGON EST MORT ★

LES ÉVÉNEMENTS

```
if (lettre_active == ' ') {  
    if (rand() % 2 == 0) {  
        lettre_active = 'f';  
        diffuser("DRAGON A FAIM ! Tapez 'f'");  
    } else {  
        lettre_active = 'h';  
        diffuser("DRAGON S'ENNUIE ! Tapez 'h'");  
    }  
}
```

LE TEMPO D'UNE MANCHE

```
[t+3..6s] tic du générateur  
[t+0.0s] "DRAGON A FAIM ! Tapez 'f'"  
[t+0.4s] Lina envoie 'f' → +10 pts  
[t+0.6s] Max envoie 'f' → -2 pts (trop lent)  
[t+~5s] tic suivant..
```

SCORING & CLASSEMENT

Trois cas, trois verdicts. Un tri à bulles. Et un GAGNANT en majuscules.

+10

BONNE LETTRE · 1ER

Le plus rapide consomme la lettre, gagne 10 pts et soigne le dragon de +20.

-5

MAUVAISE LETTRE

Tu tapes 'h' alors que le dragon a faim ? Sanction : -5 (le score reste ≥ 0).

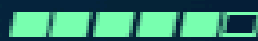
-2

TROP LENT · SPAM

Tu tapes alors qu'aucune lettre n'est active ? Petit frisson : -2 pts.

=== CLASSEMENT FINAL ===

1er LINA  **85 pts**

2eme MAX  **62 pts**

3eme NOA  **31 pts**

★ **GAGNANT : LINA - 85 PTS** ★

TRI À BULLES - server.c

```
for (i = 0; i < nb_joueurs - 1; i++)
  for (j = 0; j < nb_joueurs-1-i; j++)
    if (joueurs[j].score <
        joueurs[j+1].score) {
      Joueur tmp = joueurs[j];
      joueurs[j] = joueurs[j+1];
      joueurs[j+1] = tmp;
    }
```

```
// tous les threads sont morts ici
// → pas besoin de mutex
```

→ Le tri se fait après pthread_join : aucune concurrence, aucun verrou nécessaire. Élégant.

DÉMO — UNE PARTIE

Six étapes, du `./server` au **classement final**. Suivez le timeline.

► TIMELINE D'UNE PARTIE : de la commande shell au podium, en moins de 90 secondes.



→ Tout le code que vous avez vu... s'exécute dans cet ordre, en parallèle, sans planter.

APPORTS PÉDAGOGIQUES

Six notions de **systèmes & réseaux**, manipulées sans page de manuel — juste avec des doigts qui pianotent.



THREADS POSIX

Création (`pthread_create`), attente (`pthread_join`),
exécution concurrente. On voit, on entend, on joue.



MUTEX & CRITIQUE

Identifier les variables partagées et les protéger en **une**
seule section critique.



SOCKETS TCP

socket / bind / listen / accept côté serveur, connect
côté client. Le réseau démystifié.



RACE CONDITION

Comprendre *concrètement* pourquoi **i++** sans verrou
peut donner n'importe quoi à plusieurs.



CYCLE DE VIE

Démarrage, exécution, arrêt propre : `shutdown`,
`pthread_join`, `close`. Pas de fuite, pas de zombie.



DESIGN LOGICIEL

Séparer **qui décide** (serveur) de **qui affiche** (client). Une
vraie architecture, en mini.

MISSION ACCOMPLIE

Le dragon est mort. Les concepts, eux, sont **bien vivants**.

450

LIGNES DE C

deux fichiers, zéro dépendance externe, juste **pthread**.

N+1

THREADS COORDONNÉS

un générateur, N joueurs, un mutex pour les départager.

∞

DUELS POSSIBLES

tant qu'il y aura des claviers et des dragons à sauver.

★ LA MORALE ★

Un mutex bien placé vaut **mille printf de debug**.
Et un dragon affamé vaut **mille pages de cours** sur les threads.

